Lab 7: Fuzzing for Input Validation Bugs

Student	**Prajwal Hegde** · SRN PES1UG24CS593
Application	`Lab 7/scholarship_awards.py` (parse_applicant, marks_average, payable_award)
Assigned repo	github.com/SETAPESU26/L6_L7_44_scholarship_awards (Lab 7 folder)
Tools	Python 3.13 · Hypothesis 6.170 · pytest 9.1

Summary

Run	Buggy code	Fixed code
`python scholarship_awards.py` with invalid input	Crashes with a traceback	Prints "Invalid input..." and continues
`pytest test_scholarship_awards.py` (8 Hypothesis tests)	**7 failed**, 1 passed	**8 passed**

Bugs found by fuzzing

Smallest failing inputs reported by Hypothesis on the buggy code. Some are crashes; others are wrong results that would never crash, such as `nan` or marks over 100.

Function	Input	Result	Cause
`parse_applicant`	`''`	ValueError: not enough values to unpack	No ':' in the text
`parse_applicant`	`':'`	ValueError: could not convert string to float: ''	Empty marks after ':'
`parse_applicant`	`None`	AttributeError: 'NoneType' has no attribute 'split'	Input is not a string
`parse_applicant`	`'merit:101.0'`	No crash: returns marks 101.0	Marks over 100 accepted
`marks_average`	`''`	ValueError: invalid literal for int() ''	Empty CSV / empty mark
`marks_average`	`None`	AttributeError: 'NoneType' has no attribute 'split'	Input is not a string
`marks_average`	`'-1'`	No crash: averages -1	Negative marks accepted
`payable_award`	`''`	ValueError: not enough values to unpack	No '|' in the text
`payable_award`	`'|'`	ValueError: could not convert string to float: ''	Empty amount
`payable_award`	`None`	AttributeError: 'NoneType' has no attribute 'split'	Input is not a string
`payable_award`	`'0|nan'`	No crash: returns nan	float() accepts 'nan'; max(nan, 0) is nan
`payable_award`	`'1|1'`	No crash: returns 0.17999999999999994	Money amount not rounded

Fixes applied

Where	Fix
All three	Return `None` for input that is not a string, instead of calling `.split()` on it.

`parse_applicant`, `payable_award`	Split on ':' or '|' and require exactly two parts; strip spaces around each part.
All three	Accept numbers only as plain ASCII decimals (regex). This rejects empty parts, signs, `nan`, `inf`, exponents such as `1e999`, `1_000` and Unicode digits.
`parse_applicant`	Category must be letters (words); marks must be 0–100.
`marks_average`	Every mark must be a whole number 0–100, otherwise the whole CSV is rejected.
`payable_award`	Amounts must be non-negative; the result is rounded to 2 decimal places and never below 0.0.
`main`	Print a clear 'Invalid input' message when a function returns `None`, and stop cleanly if input ends (EOF).

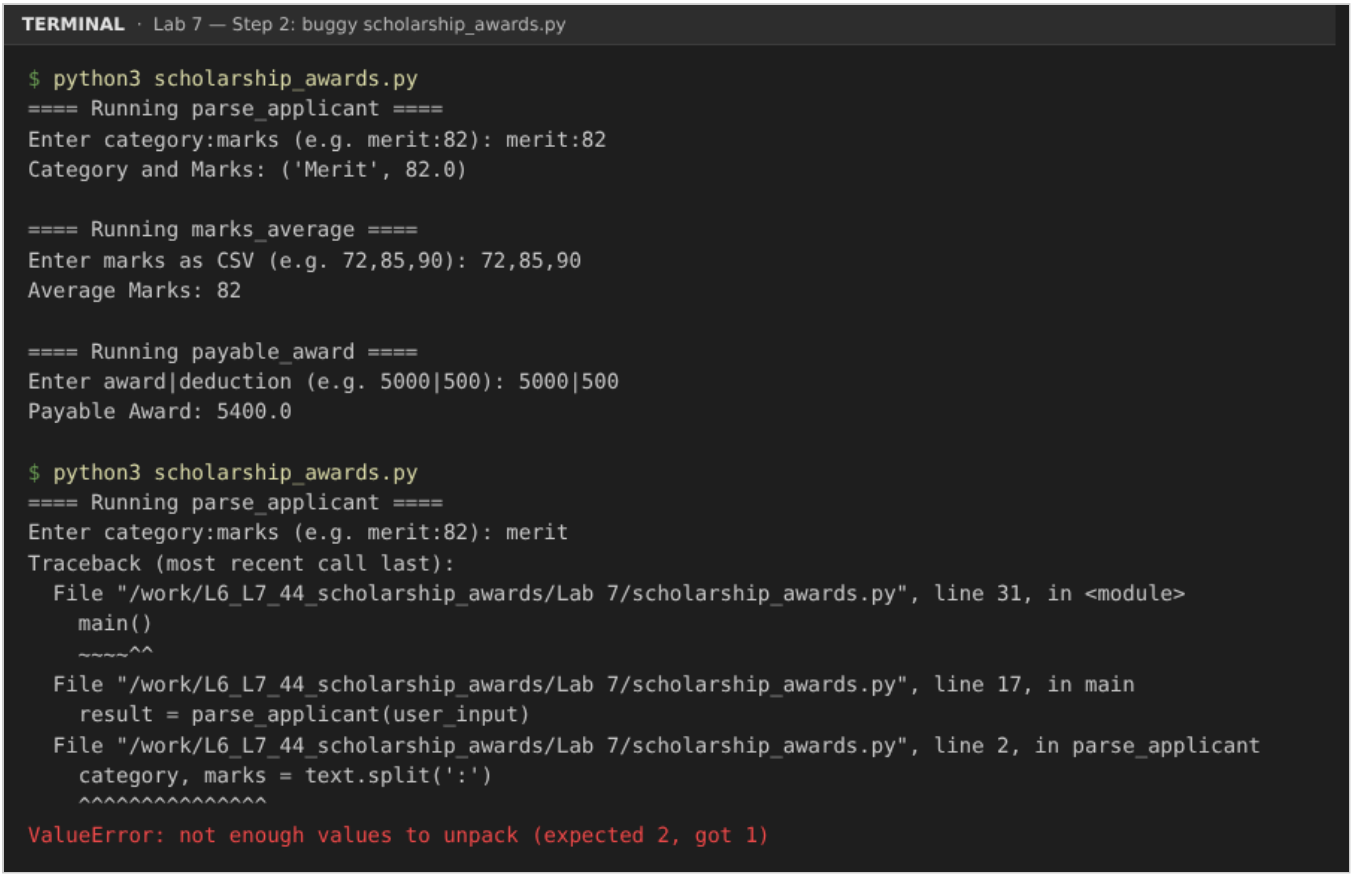
Every function now returns `None` for input it cannot interpret, instead of crashing or returning a meaningless value.

Tests written in `test_scholarship_awards.py`

- **No-crash tests** for all three functions, using `st.text() | st.none()` plus text built from the separators (':', ',', '|', '.', '-') and 500 examples each. Each test also checks the result is `None` or a sensible value.
- **Structured fuzzing:** category plus number-like text (including `nan`, `inf`, `1e999` and random floats) to check marks stay within 0–100. Random integer lists, including negatives and values over 100, for `marks_average`. Pairs of number-like text to check `payable_award` is always finite and non-negative.
- **Correct results for valid input:** category and marks round trip, the average of valid marks, and award × 1.18 − deduction rounded to 2 decimal places.

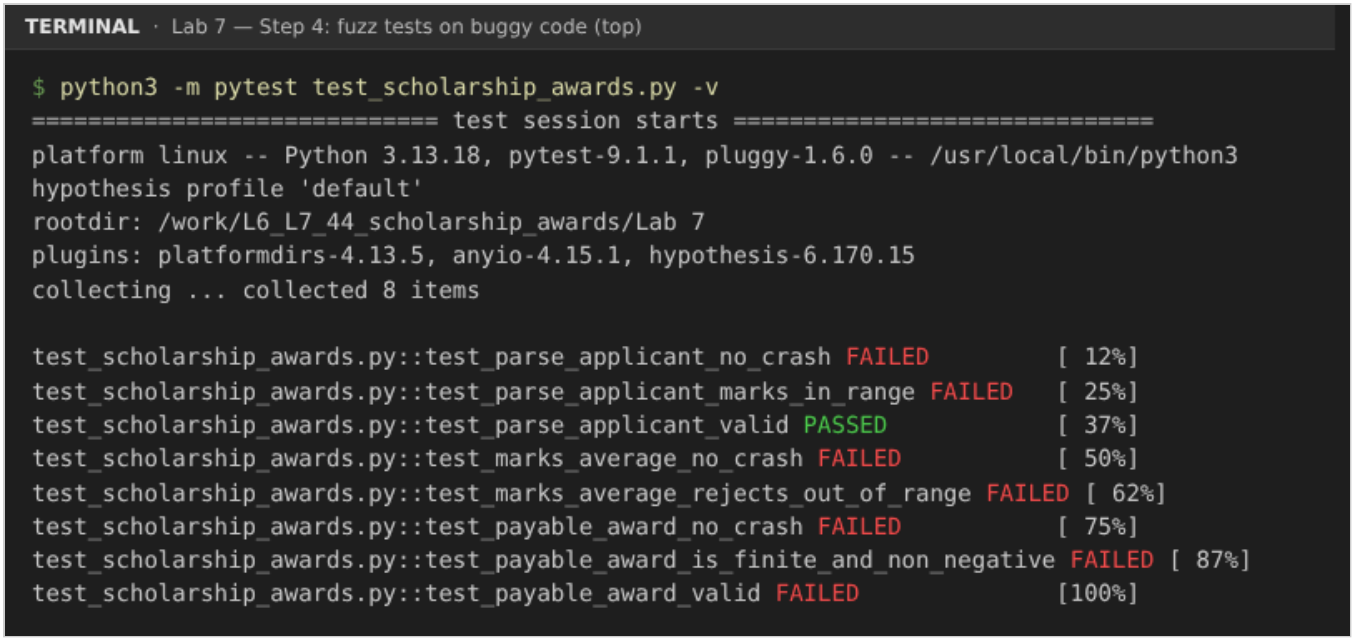
Step 2: run the buggy application

A valid run works, but entering `merit` (no ':') crashes the program.



Command: `python scholarship_awards.py`

Step 4: fuzz tests on the buggy code (failing)



Command: `python -m pytest test_scholarship_awards.py -v`: 7 of 8 tests fail.

TERMINAL · Lab 7 — Step 4: fuzz tests on buggy code (end of output)

```
    +----------------------------------
_____________________________ test_payable_award_valid ______________________________

    @given(st.integers(min_value=0, max_value=100000),
>          st.integers(min_value=0, max_value=100000))
                   ^^^

test_scholarship_awards.py:104:

award = 1, deduction = 1

    @given(st.integers(min_value=0, max_value=100000),
           st.integers(min_value=0, max_value=100000))
    def test_payable_award_valid(award, deduction):
        expected = max(round(award * 1.18 - deduction, 2), 0)
>       assert payable_award(f"{award}|{deduction}") == expected
E       AssertionError: assert 0.17999999999999994 == 0.18
E        +  where 0.17999999999999994 = payable_award('1|1')
E       Failing test case: test_payable_award_valid(
E           award=1,
E           deduction=1,
E       )

test_scholarship_awards.py:107: AssertionError
========================== short test summary info ===========================
FAILED test_scholarship_awards.py::test_parse_applicant_no_crash - ExceptionG...
FAILED test_scholarship_awards.py::test_parse_applicant_marks_in_range - Exce...
FAILED test_scholarship_awards.py::test_marks_average_no_crash - ExceptionGro...
FAILED test_scholarship_awards.py::test_marks_average_rejects_out_of_range - ...
FAILED test_scholarship_awards.py::test_payable_award_no_crash - ExceptionGro...
FAILED test_scholarship_awards.py::test_payable_award_is_finite_and_non_negative
FAILED test_scholarship_awards.py::test_payable_award_valid - AssertionError:...
======================== 7 failed, 1 passed in 2.75s =========================
```

End of the same output: `payable_award('1|1')` returns 0.17999999999999994 instead of 0.18, plus the summary.

Step 5: run the fixed application

The same valid run gives the same results; invalid input now prints a message instead of crashing.

TERMINAL · Lab 7 — Step 5: fixed scholarship_awards.py

```
$ python3 scholarship_awards.py
==== Running parse_applicant ====
Enter category:marks (e.g. merit:82): merit:82
Category and Marks: ('Merit', 82.0)

==== Running marks_average ====
Enter marks as CSV (e.g. 72,85,90): 72,85,90
Average Marks: 82

==== Running payable_award ====
Enter award|deduction (e.g. 5000|500): 5000|500
Payable Award: 5400.0

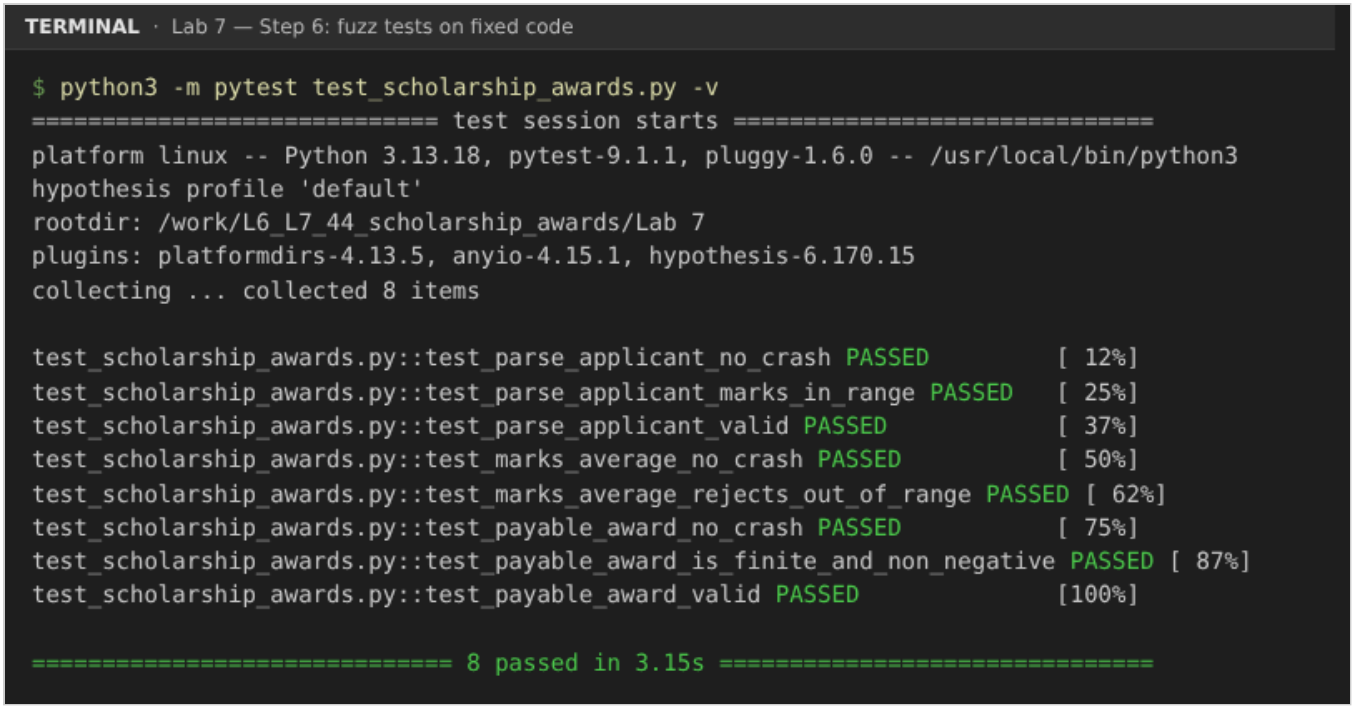
$ python3 scholarship_awards.py
==== Running parse_applicant ====
Enter category:marks (e.g. merit:82): merit
Invalid input: expected a category and marks from 0 to 100 like merit:82.

==== Running marks_average ====
Enter marks as CSV (e.g. 72,85,90): 72,,90
Invalid input: expected whole marks from 0 to 100 separated by commas like 72,85,90.

==== Running payable_award ====
Enter award|deduction (e.g. 5000|500): 5000|nan
Invalid input: expected two non-negative amounts like 5000|500.
```

Command: `python scholarship_awards.py`

Step 6: fuzz tests on the fixed code (passing)



Command: `python -m pytest test_scholarship_awards.py -v`: all 8 tests pass.